

Real-Time Social Media Trend Analysis

A CSC 258 Distributed Systems Project

Overview

This project is a real-time distributed social media trend detection system built using Kafka, Docker, and Google Cloud. The system ingests live social media posts from Bluesky Jetstream, processes trending terms and phrases, stores trend snapshots, and displays the latest trends on a live dashboard.

The architecture follows a distributed microservices design where services communicate independently using Kafka and REST APIs. The system is containerized using Docker and deployed on a Google Cloud Virtual Machine.

Team Members

Team Member Contribution

Soulius Jones Survey Paper Writer ,Code Reviewer Report writer

Abdurrehman Aslam Code Developer and Survey Paper Reviewer

Abubaker Sayyed Survey Paper Writer, Cloud Deployment,Code Reviewer,
containerization and report Writing

Fidel Serrano Code Developer and Survey Paper Reviewer

System Architecture

The project uses a distributed microservices architecture with the following pipeline:

ingestion → kafka-broker → processing → storage → dashboard

Services

Ingestion Service

Collects live social media posts from Bluesky Jetstream using WebSocket connections and normalizes incoming data into structured JSON format.

Kafka Broker

Acts as the distributed message broker between ingestion and processing services. Kafka provides buffering, asynchronous communication, scalability, and fault tolerance.

Processing Service

Consumes posts from Kafka, extracts trending words and phrases, filters noisy data using stopwords, and generates trend snapshots and example posts.

Storage Service

Stores trend snapshots and example posts in JSON format for dashboard visualization.

Dashboard Service

Displays live trending terms and example posts through a web-based dashboard.

Code Structure

services/

|

|— broker/

| └─ config.py

```
|
|
|— ingestion/
|   |— consumer.py
|   |— producer.py
|   |— normalize.py
|   |— main.py
|   └— config.py
|
|
|— processing/
|   |— consumer.py
|   |— processor.py
|   |— main.py
|   └— config.py
|
|
|— storage/
|   |— trend_save.py
|   |— config.py
|   |— unwanted_words.py
|   └— logs/
|       |— trends.json
|       └— example_posts.json
|
|
└— dashboard/
    |— index.html
    |— styles.css
    └— script.js
```

Technologies Used

Python

Apache Kafka

Docker

Docker Compose

Flask

HTML / CSS / JavaScript

Google Cloud Platform

PostgreSQL (future storage support)

Dependencies and Environment

Component	Technology
-----------	------------

Programming Language	Python 3.11
----------------------	-------------

Container Runtime	Docker
-------------------	--------

Orchestration	Docker Compose
---------------	----------------

Message Broker	Apache Kafka
----------------	--------------

Cloud Platform	Google Cloud VM
----------------	-----------------

Dashboard	HTML/CSS/JavaScript
-----------	---------------------

Shell	PowerShell / Linux Bash
-------	-------------------------

Running Locally

1. Start Kafka Broker

```
docker compose up -d broker
```

2. Run Ingestion Service

```
python -m services.ingestion.main
```

3. Run Processing Service

```
python -m services.processing.main
```

4. Start Local HTTP Server

```
python -m http.server 8000
```

5. Open Dashboard

```
http://localhost:8000/services/dashboard/index.html
```

Running on Cloud

1. Open Google Cloud VM

Google Cloud Console → Compute Engine → VM Instances → SSH

2. Navigate to Project Directory

```
cd ~/CSC_258_Project-kafka-implementation-2.1
```

3. Start Docker Services

```
docker-compose up -d --build
```

This command starts:

zookeeper

kafka-broker

ingestion

processing

storage

dashboard

4. Verify Running Containers

```
docker-compose ps
```

Expected services:

zookeeper

kafka-broker

ingestion

processing

dashboard

5. Restart Services if Kafka Starts Slowly

docker-compose restart ingestion processing

6. View Processing Logs

docker-compose logs -f processing

Expected output:

Processed X posts

Saved trend snapshot

Saved example posts

Dashboard Access

Live Dashboard:

<http://34.58.166.229:8080>

Firewall Configuration

The following firewall ports must be enabled in Google Cloud:

Port	Purpose
------	---------

22	SSH Access
----	------------

8080	Dashboard Access
------	------------------

9092	Kafka Broker
------	--------------

Cloud Deployment Features

Public dashboard access through VM external IP

Docker containerized services

Kafka-based distributed messaging

Real-time dashboard updates

Independent microservices architecture

Scalability

The system supports scalability through:

Kafka-based service decoupling

Independent microservices

Docker containerization

Cloud deployment

Kubernetes-ready architecture for future scaling

Kafka allows ingestion and processing services to scale independently without direct coupling.

Fault Tolerance

The system improves fault tolerance through:

Kafka message buffering

Independent service restarts

WebSocket reconnection logic

Atomic JSON file writing

Docker container isolation

If processing temporarily fails, Kafka retains messages until consumers recover.

Security

The system includes:

Environment-based configuration

Input validation

Service isolation using containers

Separation of storage and processing logic

Other Useful Commands

Install Python Dependencies

```
pip install -r requirements.txt
```